

Лабораторная работа №1.

Аутентификация пользователей

Тема: Регистрация и вход пользователей в Express-приложении

Цель: Создать модуль аутентификации Internet Shop с регистрацией и входом пользователя, безопасным хранением passwordHash и разделением Route, Controller и Service.

Дисциплина: Node.js и Express.js

Исходные условия

Продолжайте работу с проектом `express-shop`. В проекте уже используются Express.js, PostgreSQL и Prisma Client.

На этой лабораторной `argon2.hash()` и `argon2.verify()` используются как готовые операции. Подробно хеширование паролей будет изучено на следующем занятии.

Безопасность

Не сохраняйте пароль пользователя в открытом виде даже в учебном проекте. В таблице `users` должен храниться только `passwordHash`.

1. Установите argon2

```
PowerShell  
  
npm install argon2
```

2. Обновите модель User

```
prisma/schema.prisma  
  
model User {  
  id          Int      @id @default(autoincrement())  
  name        String  
  email       String   @unique  
  passwordHash String   @map("password_hash")  
  role        String   @default("user")  
  orders      Order[]  
  
  @@map("users")  
}
```

Поле `email` используется как идентификатор пользователя. Поле `passwordHash` предназначено для результата безопасного password hashing. Поле `role` понадобится в теме авторизации.

3. Создайте миграцию

```
PowerShell
```

```
npx prisma@7 migrate dev --name add_user_auth_fields
npx prisma@7 generate
```

4. Создайте passwordService

```
services/passwordService.js

import * as argon2 from "argon2";

export async function hashPassword(password) {
  return argon2.hash(password);
}

export async function verifyPassword(passwordHash, password) {
  return argon2.verify(passwordHash, password);
}
```

На этой работе

Не изменяйте параметры Argon2. Сейчас задача — понять общий Authentication flow. Алгоритм и настройки password hashing будут разобраны отдельно.

5. Создайте Auth Service

```
services/authService.js

import { prisma } from "../config/prisma.js";
import { hashPassword, verifyPassword } from "../passwordService.js";

export async function registerUser({ name, email, password }) {
  const existing = await prisma.user.findUnique({
    where: { email }
  });

  if (existing) {
    throw new Error("Email already registered");
  }

  const passwordHash = await hashPassword(password);

  return prisma.user.create({
    data: {
      name,
      email,
      passwordHash
    },
    select: {
      id: true,
      name: true,
      email: true,
      role: true
    }
  });
}

export async function authenticateUser(email, password) {
```

```

const user = await prisma.user.findUnique({
  where: { email }
});

if (!user) {
  return null;
}

const valid = await verifyPassword(
  user.passwordHash,
  password
);

if (!valid) {
  return null;
}

return {
  id: user.id,
  name: user.name,
  email: user.email,
  role: user.role
};
}

```

`registerUser()` создаёт новую учётную запись. `authenticateUser()` проверяет существующую. Ни одна функция не возвращает `passwordHash` клиенту.

6. Создайте Auth Controller

controllers/authController.js

```

import {
  registerUser,
  authenticateUser
} from "../services/authService.js";

export async function register(req, res, next) {
  try {
    const user = await registerUser(req.body);
    res.status(201).json({ user });
  } catch (error) {
    next(error);
  }
}

export async function login(req, res, next) {
  try {
    const { email, password } = req.body;
    const user = await authenticateUser(email, password);

    if (!user) {
      return res.status(401).json({
        error: "Invalid email or password"
      });
    }

    res.json({ user });
  } catch (error) {
    next(error);
  }
}

```

```
}
```

7. Создайте Auth routes

```
routes/authRoutes.js

import { Router } from "express";
import {
  register,
  login
} from "../controllers/authController.js";

const router = Router();

router.post("/register", register);
router.post("/login", login);

export default router;
```

8. Подключите маршруты

```
app.js

import authRoutes from "./routes/authRoutes.js";

app.use("/api/auth", authRoutes);
```

Auth endpoints

Метод	URL	Назначение
POST	/api/auth/register	Создать учётную запись
POST	/api/auth/login	Проверить email и password

9. Проверьте регистрацию

```
POST /api/auth/register

POST /api/auth/register
Content-Type: application/json

{
  "name": "Alex",
  "email": "alex@example.com",
  "password": "ExamplePassword123!"
}
```

Пример ответа

```
{
```

```
"user": {
  "id": 1,
  "name": "Alex",
  "email": "alex@example.com",
  "role": "user"
}
```

Значение `passwordHash` не должно присутствовать в HTTP-ответе.

10. Проверьте данные в PostgreSQL

Проверка users

```
SELECT
  id,
  name,
  email,
  password_hash,
  role
FROM users
ORDER BY id;
```

В `password_hash` должна находиться длинная строка Argon2, а не исходный пароль `ExamplePassword123!`.

11. Проверьте успешный login

POST /api/auth/login

```
POST /api/auth/login
Content-Type: application/json

{
  "email": "alex@example.com",
  "password": "ExamplePassword123!"
}
```

Пример ответа

```
{
  "user": {
    "id": 1,
    "name": "Alex",
    "email": "alex@example.com",
    "role": "user"
  }
}
```

12. Проверьте неверный password

```
POST /api/auth/login
Content-Type: application/json
```

```
{
  "email": "alex@example.com",
  "password": "wrong-password"
}
```

```
{
  "error": "Invalid email or password"
}
```

13. Проверьте неизвестный email

POST /api/auth/login
Content-Type: application/json

```
{
  "email": "unknown@example.com",
  "password": "ExamplePassword123!"
}
```

```
{
  "error": "Invalid email or password"
}
```

Одинаковая внешняя ошибка

Для неизвестного email и неправильного password клиент должен получать одинаковое сообщение. Не раскрывайте существование учётной записи через login endpoint.

14. Проверьте повторную регистрацию

Повторите POST /api/auth/register с уже существующим email. PostgreSQL и Service не должны позволить создать вторую учётную запись с тем же email.

15. Подключите frontend

Пример login на frontend

```
const response = await fetch("http://localhost:3000/api/auth/login", {
  method: "POST",
  headers: {
    "Content-Type": "application/json"
  },
  body: JSON.stringify({ email, password })
});

const data = await response.json();

if (response.ok) {
  console.log("Authenticated user:", data.user);
} else {
  console.error(data.error);
}
```

```
}
```

Пока без session или JWT

На этой лабораторной login только проверяет credentials. Механизм сохранения аутентифицированного состояния будет добавлен на следующих занятиях.

Самостоятельное задание

1. Добавьте validation для `name`, `email` и `password` при регистрации.
2. Добавьте validation для `email` и `password` при login.
3. Запретите регистрацию пустого `name`.
4. Запретите регистрацию с повторяющимся `email`.
5. Проверьте, что ни один HTTP-ответ не содержит `passwordHash`.
6. Создайте простую HTML-форму login и отправьте данные через `fetch()`.
7. Выведите имя пользователя на странице после успешного login.
8. Покажите единое сообщение об ошибке при неверных credentials.

Требования к результату

- В User существует поле `passwordHash`.
- Исходный password не сохраняется в PostgreSQL.
- Регистрация работает через `POST /api/auth/register`.
- Login работает через `POST /api/auth/login`.
- `passwordHash` отсутствует в ответах API.
- Неверный password возвращает 401.
- Неизвестный email возвращает такое же внешнее сообщение login.
- Auth Service не использует `req` и `res`.
- Auth Controller не содержит Prisma-запросов.
- Frontend может отправить login через `fetch()`.

Критерии оценивания

№	Критерий
1	argon2 установлен и импортируется
2	User содержит passwordHash
3	email имеет уникальное ограничение
4	Миграция успешно применена
5	register создаёт User
6	В PostgreSQL не хранится открытый password
7	passwordHash отсутствует в register response
8	login с правильным password успешен
9	login с неправильным password возвращает 401

№	Критерий
10	Неизвестный email возвращает одинаковую login-ошибку
11	passwordHash отсутствует в login response
12	Prisma-запросы расположены в Service
13	Controller отвечает только за HTTP
14	Frontend выполняет login через fetch()
15	Студент объясняет различие Authentication и Authorization